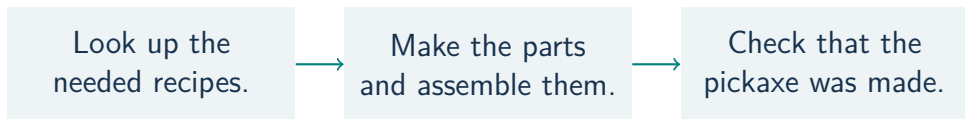


How can a model solve an unfamiliar task?

We want a language model to discover useful steps, use tools, and check whether the job is finished.

A simple test: make a pickaxe in a crafting game.



The game checks actual success, not whether the answer sounds convincing.

The larger idea is a **recursive language model (RLM)**: a model that can use code and ask helper models to solve smaller parts.

Simplified example. Our recent crafting experiments use one model with tools; they do not yet test recursive helper calls.

We tried learning from success and failure.

In **reinforcement learning (RL)**, the model tries answers or actions, receives a score, and learns from the feedback.

We saw some gains, but not reliable progress in planning.

Some models answered or selected information better. The gains often weakened on new examples, or did not clearly beat more learning from worked examples.

One possible difficulty: A model can make wrong parts, recover, and finish the pickaxe. A reward for finishing does not identify which earlier actions helped.

Open question: How can we reward useful decisions along the way?

The pickaxe story illustrates why final success may not identify helpful steps. It is not a proven explanation of all our RL results.

Two recent findings suggest a useful division of work.

Teach the model how to find the steps.

We trained models to imitate worked examples. Examples from a program that discovered recipes worked better than examples from one that already knew them.

Both programs showed recipe lookups; they differed in how they found the sequence of actions.

Let code handle exact details.

The model still chose what to make. When code filled in ingredients from an already-read recipe, more tasks succeeded—without extra model training.

Choosing a useful step and executing it correctly are different problems.

Preliminary results in one crafting game. The teaching examples differ in several ways, so the reason for their advantage is not yet isolated.

Can better tools make learning easier?

Our working idea is to let code handle exact details so that learning can focus on choosing useful steps.

Next, test the idea on new problems. Does code help both kinds of trained model? Does the advantage from discovery examples remain?

Then return to RL. Does this clearer division of work make it easier to learn from success and failure?

Longer term: Learn when to ask a helper and when to break a task into smaller parts again.

For discussion

What other multi-step task would make this a useful and convincing result?

Broader tests are queued; improved RL and learned helper use remain open questions.